Canary Deployments

ship to 1% first, let the metrics decide

Watch the 4-min video

In one line: send a small, growing slice of real traffic to the new version, compare it with the stable baseline on errors, latency, saturation and business metrics at each step, and let automation promote or roll back, so a hidden problem hits 1% of users for minutes instead of everyone.

1 Offline isn't production

Your team trained a new ranking model and the offline metrics look great. But production has real users, real traffic and real edge cases. Ship it to everyone and a hidden problem hits everyone at once. A canary, named after the canary in the coal mine, lets a small group find out first.

2 Traffic steps

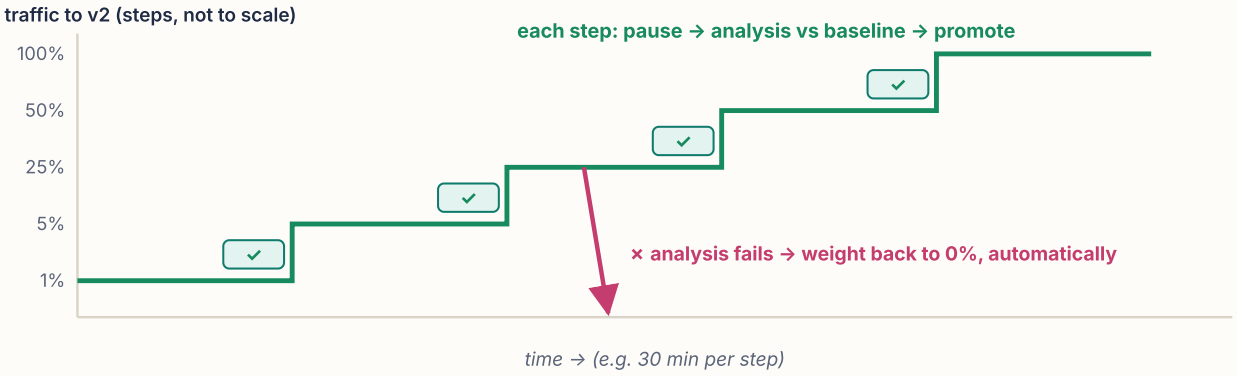


Figure 1. Progressive delivery: 1% → 5% → 25% → 50% → 100%, with a pause and an automated analysis at each step. A failed check snaps traffic back to stable.

Start at **1%**. If healthy, move to 5, 25, 50, then 100%. Each step is a chance to stop before the damage spreads. Weights are set in a router that can split by percentage: a service mesh (lesson 27), an ingress controller, or a cloud load balancer with weighted targets. Without one, Kubernetes can only approximate weights by replica counts (1 canary pod of 10 = 10%).

3 Is the canary healthy?

Don't eyeball dashboards. Compare the canary with the **stable baseline, at the same time, under the same traffic**, so time-of-day and traffic mix cancel out. Some setups (e.g. Spinnaker's Kayenta) start a fresh baseline of the old version at the same size as the canary, so both are equally new.

Signal	Example check	Why
Error rate	canary 5xx ratio < 1% and not worse than baseline	outright breakage
Latency	p99 < 300 ms, within 10% of baseline	slow code, N+1 queries
Saturation	CPU, memory, pool use per request	leaks, heavier code
Business	conversion, click-through vs baseline	"works" but worse for users

4 Automate the decision

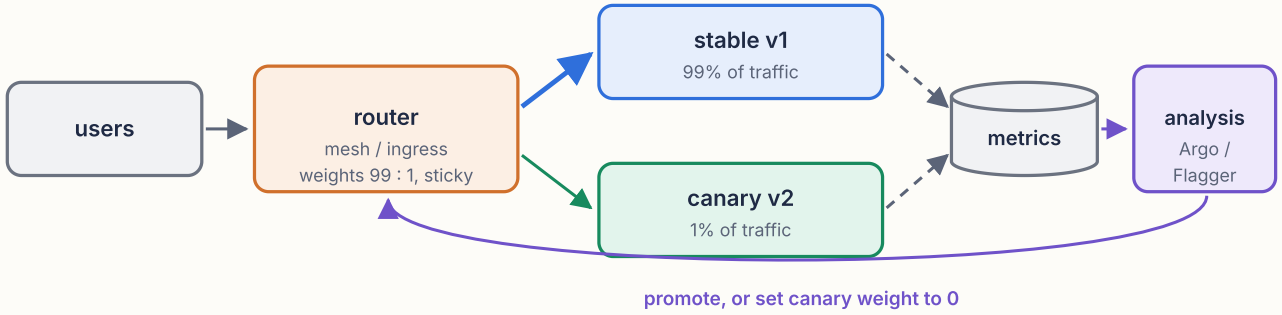


Figure 2. The router splits traffic; both versions report metrics; the analysis controller promotes or aborts. No human is needed for a rollback.

Argo Rollouts and **Flagger** run canaries on Kubernetes: you declare steps and success criteria, they shift traffic via a mesh or ingress and query Prometheus. Cloud platforms offer weighted routing too. Every step, metric and decision is logged, so you can see exactly why a rollout stopped. Rollback becomes a routine non-event, not an incident.

```
strategy:
  canary:
    steps:
    - setWeight: 1
    - pause: {duration: 30m}
    - setWeight: 5
    - pause: {duration: 30m}          # ... 25, 50, then full promotion
    analysis:                         # runs in the background during the steps
      templates: [{templateName: canary-health}]   # error rate < 1%, p99 < 300 ms, CTR vs baseline
```

Figure 3. An Argo Rollouts canary from the video: weights and pauses, with a background analysis that aborts the rollout when a metric fails.

PITFALLS

- **Too little traffic:** at 10 req/s, 30 minutes is ~18,000 requests; a rare error may not show up. Wait longer or use a bigger step.
- **Non-sticky routing:** one user bouncing between versions sees inconsistent results; hash on user ID or a cookie.
- **Compatibility:** both versions run together, so schemas and APIs must be backward compatible (lesson 73).
- **Only technical metrics:** a model can be fast and error-free yet worse for users; include business metrics.

WORKED EXAMPLE: A NEW RANKING MODEL TO 1% OF USERS

The router sends **1%** of search requests to the new model, sticky per user, and keeps 99% on the old one. Analysis compares error rate, p99 latency and click-through rate with the baseline. After **30 minutes**, latency and errors are fine, but **click-through is 3% lower**. The analysis fails, the rollout aborts automatically, and all traffic returns to the old model. Only 1% of users ever saw it, for half an hour; the team investigates offline.

5 Key terms

Canary: new version on a small slice of real traffic

Baseline: stable version measured at the same time for comparison

Progressive delivery: automated step-by-step rollout with analysis

Traffic weight: share of requests routed to a version

Blast radius: how many users a bad change can affect

Abort: automatic return of all traffic to stable

6 Interview questions

Q1. How do you decide a canary is healthy? BASIC

Compare it with the stable baseline running at the same time under the same traffic, on error rate, latency percentiles, saturation and business metrics, with enough traffic and automated pass/fail thresholds.

Q2. Why canary instead of blue-green? BASIC

It limits the blast radius to a small slice of users and catches problems only real traffic reveals; blue-green switches everyone at once.

Q3. A service gets 50 req/s. Is a 1% canary for 10 minutes enough? SCENARIO

No: that's about 300 requests, too few to detect a small error or latency change. Use a larger step (10–25%) or a longer pause.

Q4. Canary errors look slightly higher at 3 a.m. than the baseline was at noon. Problem? SCENARIO

Not a fair comparison: traffic mix differs by time. Compare canary and baseline over the same window.

Q5. Can you canary a database schema change? TRICKY

Not directly: the database is shared, so the schema change must be backward compatible and applied before (expand) or after (contract) the canary of the code.

Q6. Why can't Kubernetes alone do a 1% canary? TRICKY

A plain Service balances across pods, so weights follow replica counts; 1% needs 99 stable pods per canary pod, or a mesh/ingress that splits by percentage.

Next: SD-N-018 · Feature Flags backendroadmap.com